

软件安全实验二

班级：2022211801 学号：2022211570 姓名：项枫

一、目标

- 1、理解程序堆栈调用关系
- 2、分析堆栈溢出的基本原理
- 3、实际动手练习堆栈溢出利用方法

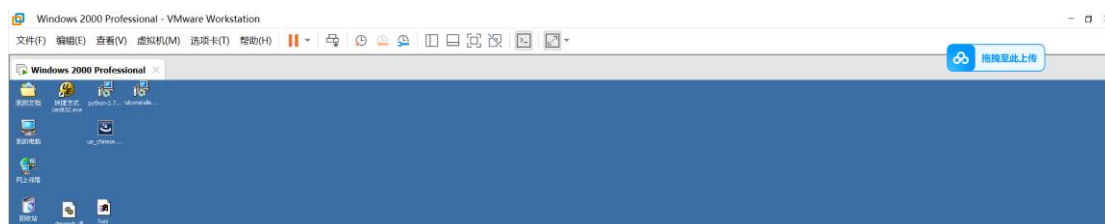
二、测试步骤与结果

（一）淹没邻接变量

1、创建可执行程序

（1）进入虚拟机

打开 VM workstation，点击文件->打开->选择我们要使用的 WIN2000 的 VMX 文件，之后我们就进入了虚拟机。



（2）研究代码

使用桌面的 VC6 打开文件夹“实验代码\stackvar”中的 main.cpp，对代码进行研究。

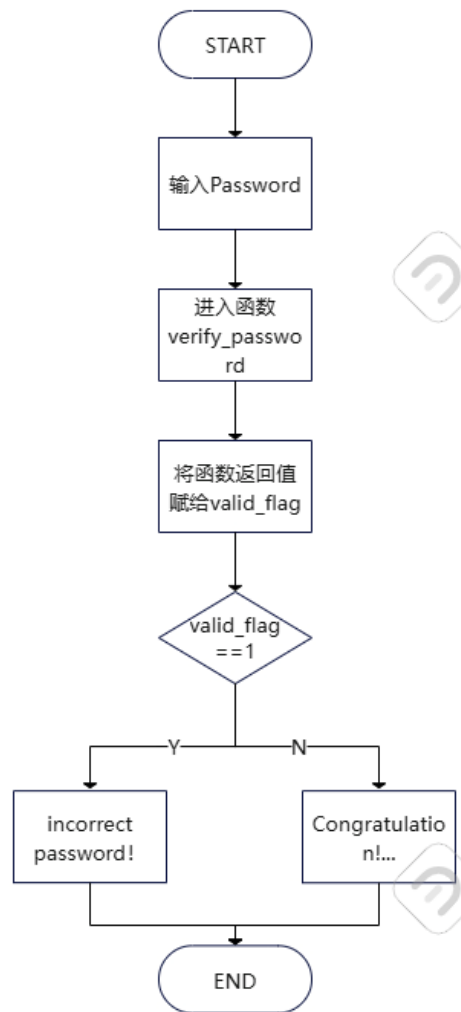
```
Microsoft Visual C++ - [main.cpp]
File Edit View Insert Project Build Tools Window Help
GetMemory

#include <stdio.h>
#include <string>
#define PASSWORD "1234567"

int verify_password (char *password)
{
    int authenticated;
    char buffer[8]; // add local buff
    authenticated=strcmp(password,PASSWORD);
    strcpy(buffer,password); //over flowed here!
    return authenticated;
}

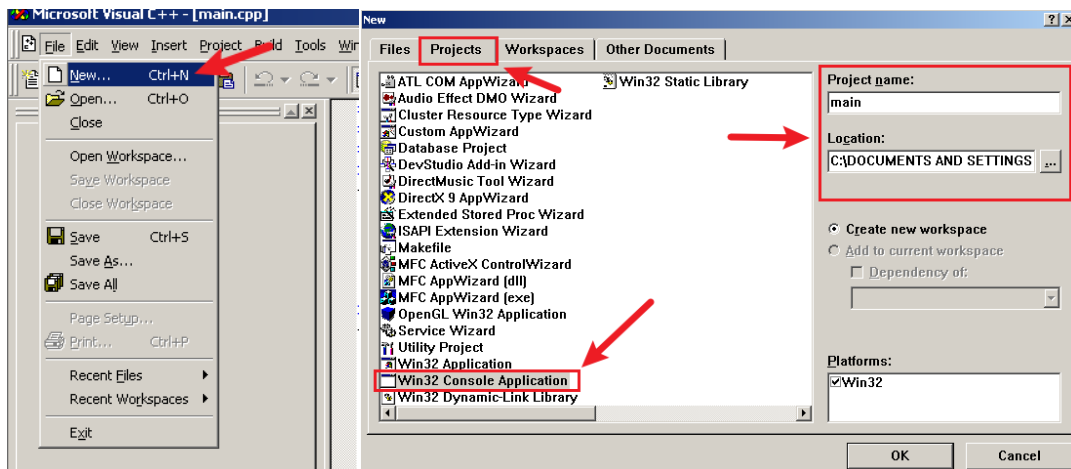
main()
{
    int valid_flag=0;
    char password[1024];
    while(1)
    {
        printf("please input password:    ");
        scanf("%s",password);
        valid_flag = verify_password(password);
        if(valid_flag)
        {
            printf("incorrect password!\n\n");
        }
        else
        {
            printf("Congratulation! You have passed the verification!\n");
            break;
        }
    }
}
```

显而易见，这是一个验证输入密码是否正确的简单小程序，代码逻辑流程图如下：

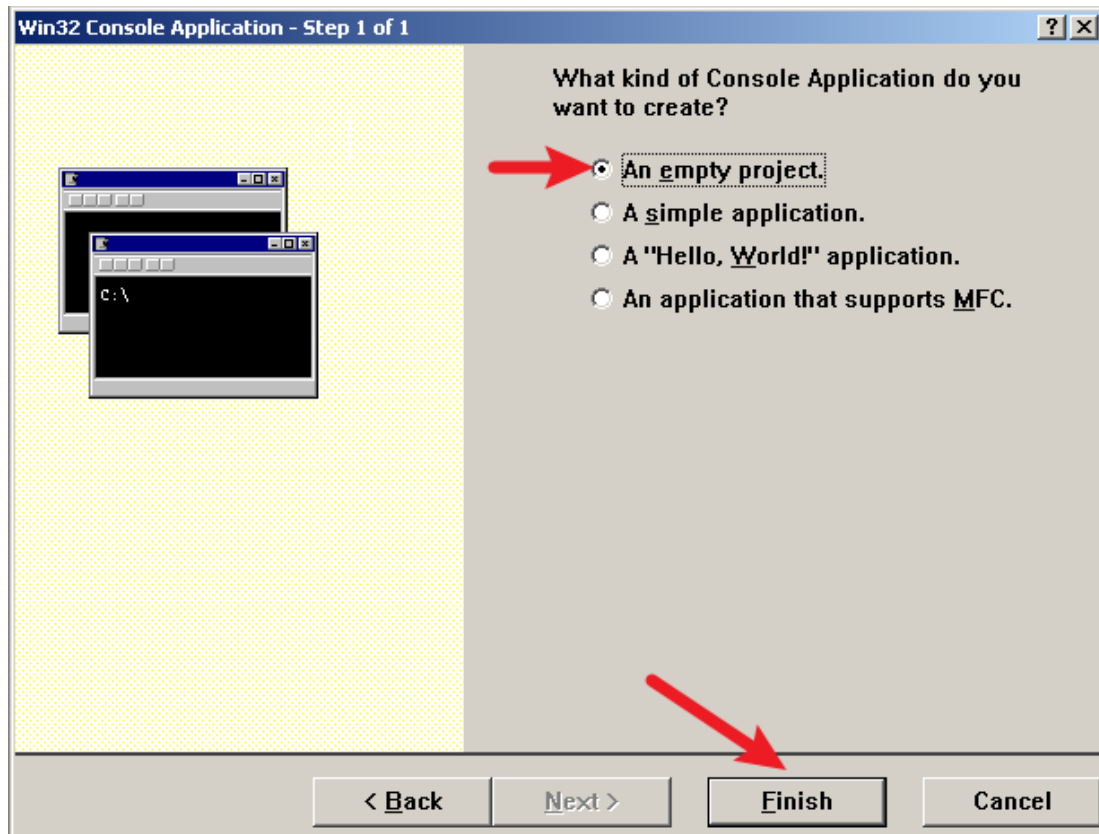


(3) 生成工程

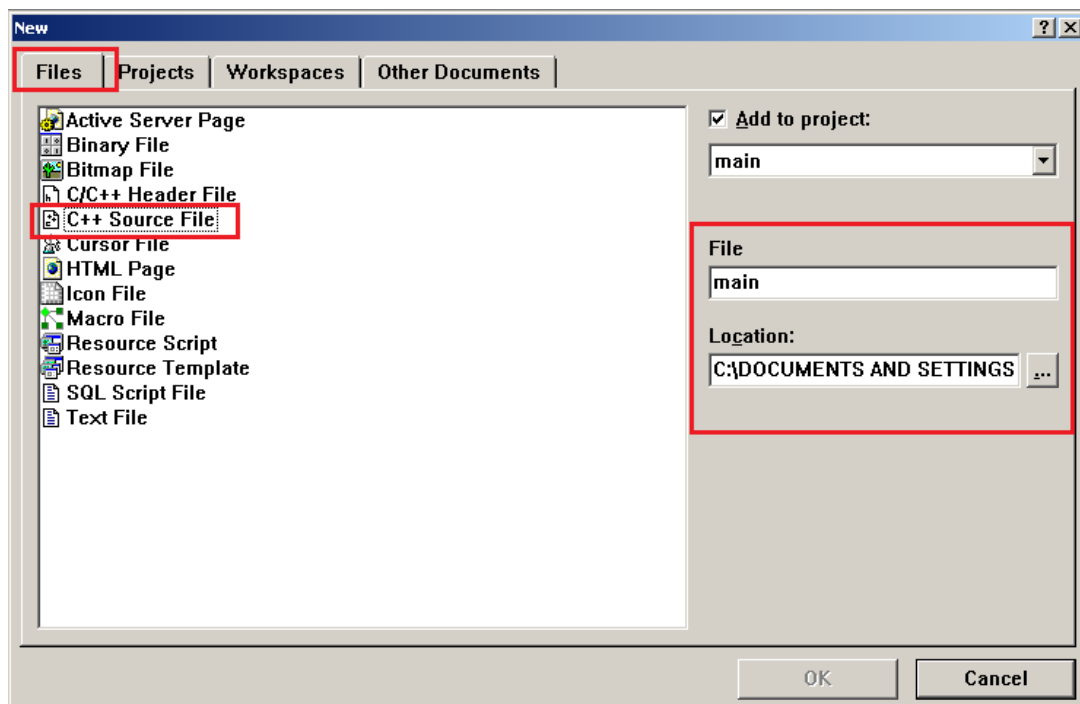
①在 VC 中，点击 File->new->Projects->Win32 Console Application，右上角设置好 project 的名称和存放路径。



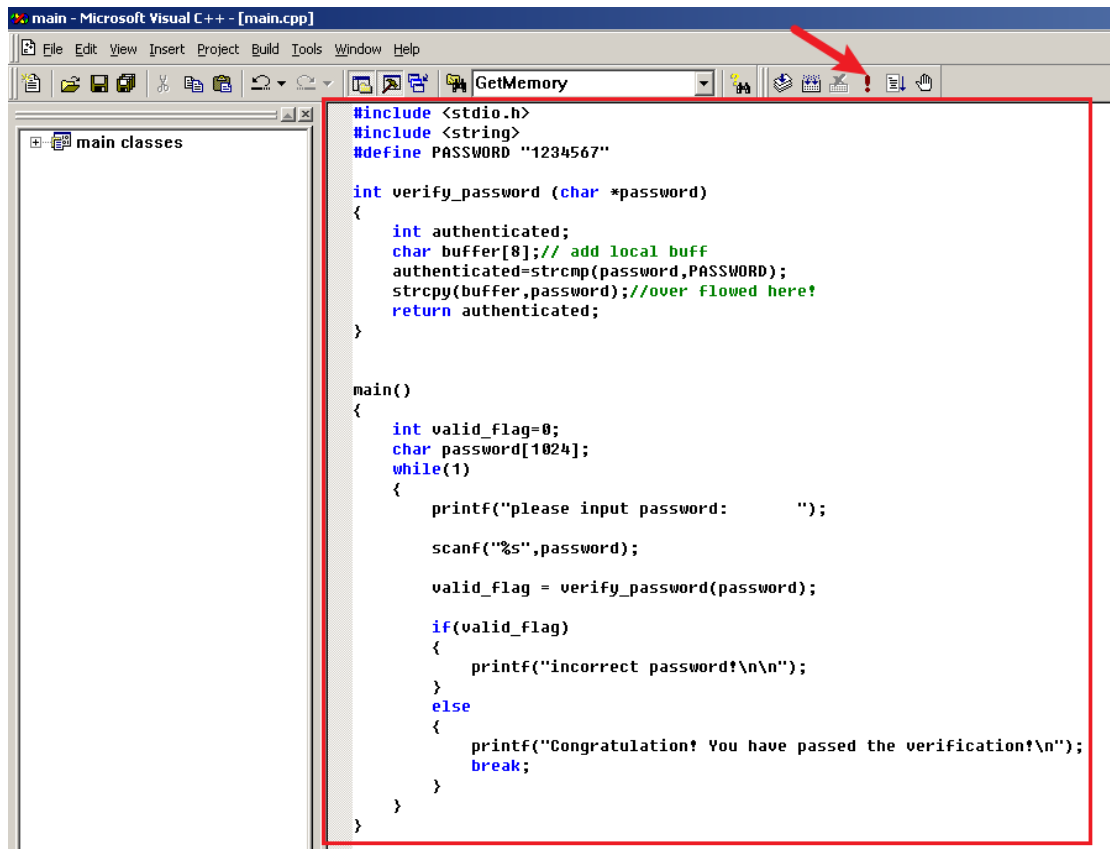
②点击 OK->An empty project->Finish



③之后再 new->Files->C++ Source File, 右侧设定好添加到的项目(我们刚才新建的项目), 文件名和路径。



④完成新建文件后, 在新建的文件下编写或复制刚才我们看到的 main 程序, 然点击右上的红色感叹号执行。



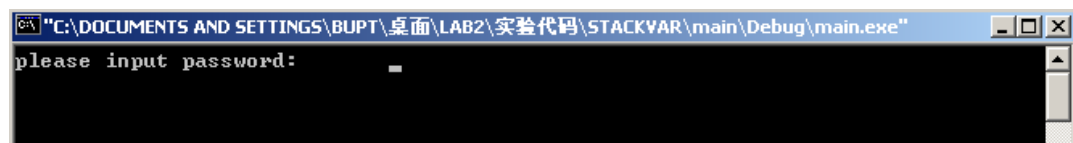
```
#include <stdio.h>
#include <string>
#define PASSWORD "1234567"

int verify_password (char *password)
{
    int authenticated;
    char buffer[8]; // add local buff
    authenticated=strcmp(password,PASSWORD);
    strcpy(buffer,password); //over flowed here!
    return authenticated;
}

main()
{
    int valid_flag=0;
    char password[1024];
    while(1)
    {
        printf("please input password:  ");
        scanf("%s",password);

        valid_flag = verify_password(password);

        if(valid_flag)
        {
            printf("incorrect password!\n\n");
        }
        else
        {
            printf("Congratulation! You have passed the verification!\n");
            break;
        }
    }
}
```



2、测试

(1) 简单测试

在运行程序后输入简单的密码，注意不要超过 7 位。

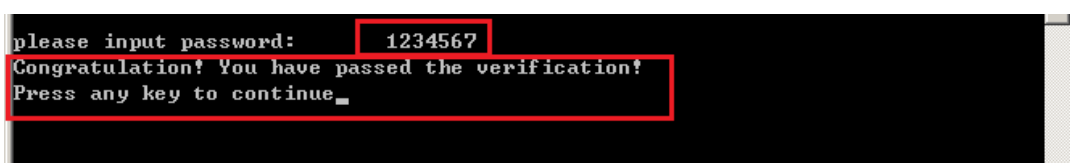
①输入错误的密码

输入错误的密码“111”，提示“incorrect password!”即密码错误。



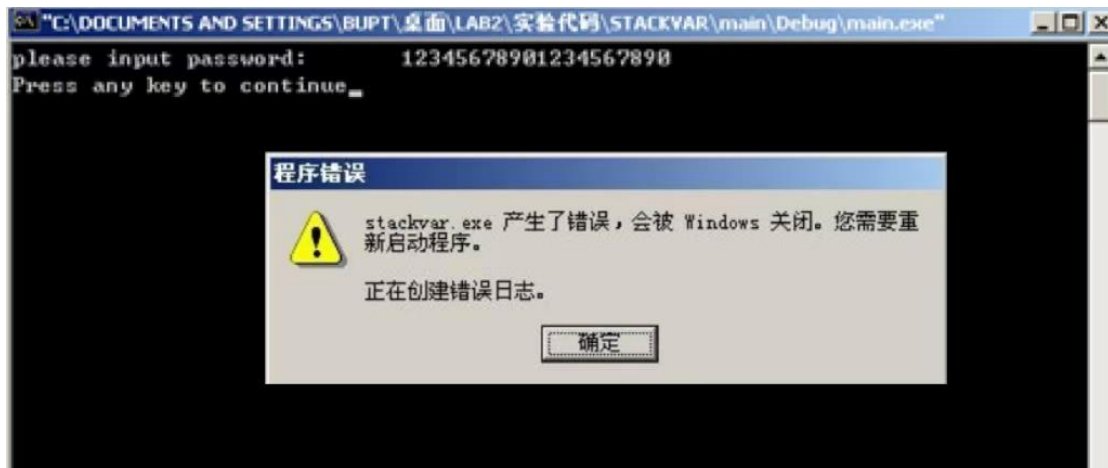
②输入正确的密码

输入正确密码“1234567”，提示“Congratulation!...”即密码正确，并结束程序。



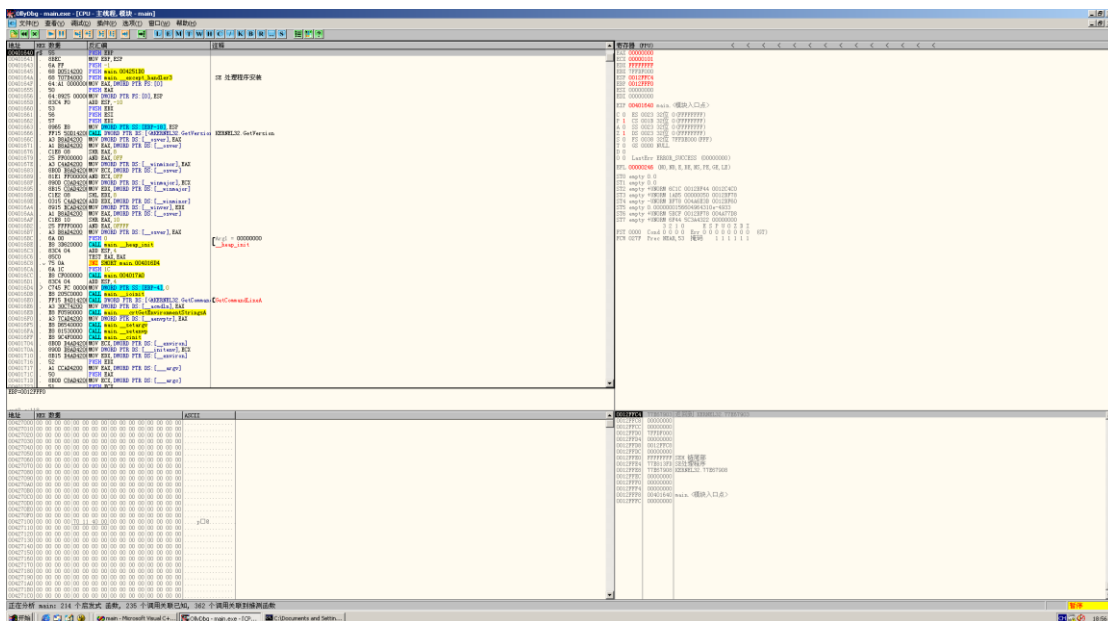
(2) 溢出测试

填写多位密码“12345678901234567890”（20 位），出现了溢出错误。



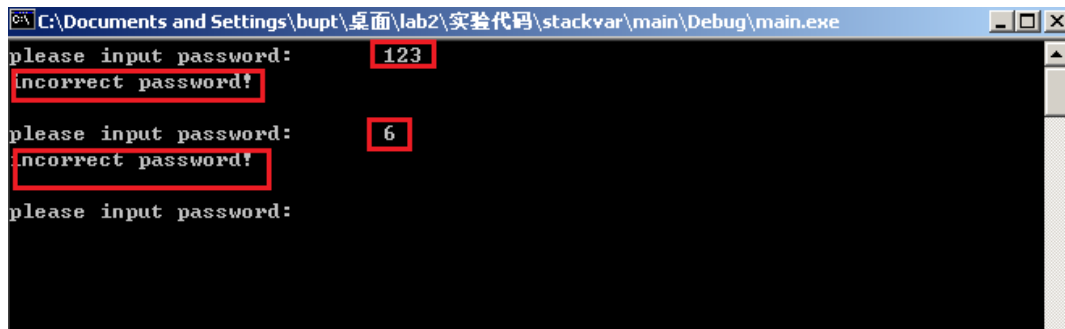
3、调试

(1) 用 ollydbg 打开 main.exe 文件



(2) 运行

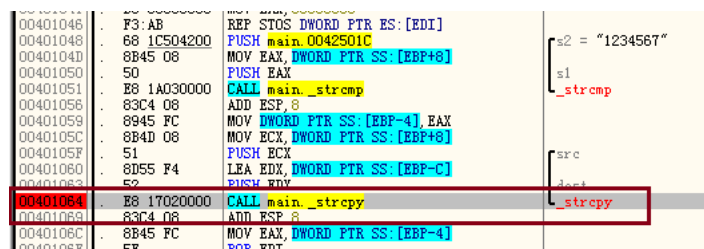
输入几个短的错误的密码（例如“123”、“6”）。



(3) 设置断点，单步运行查看数据

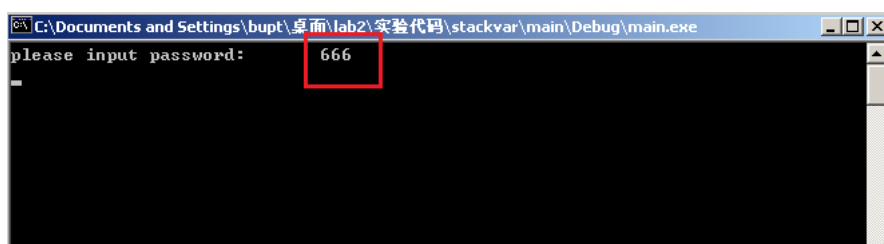
①设置断点

找到 strcpy 语句，并在这个语句上按 F2 设置断点。



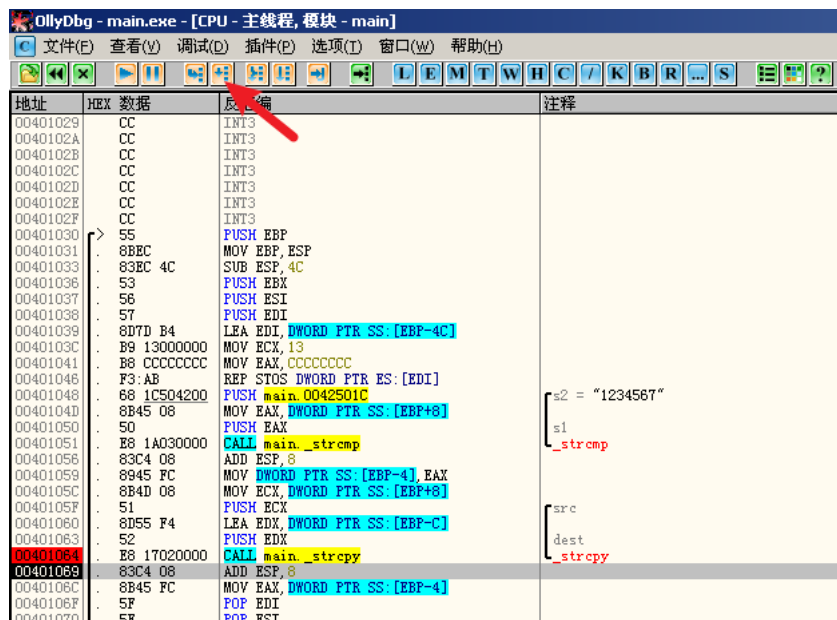
②运行

然后点击运行（程序走到这个语句时会自动停止），输入较短的密码“666”。



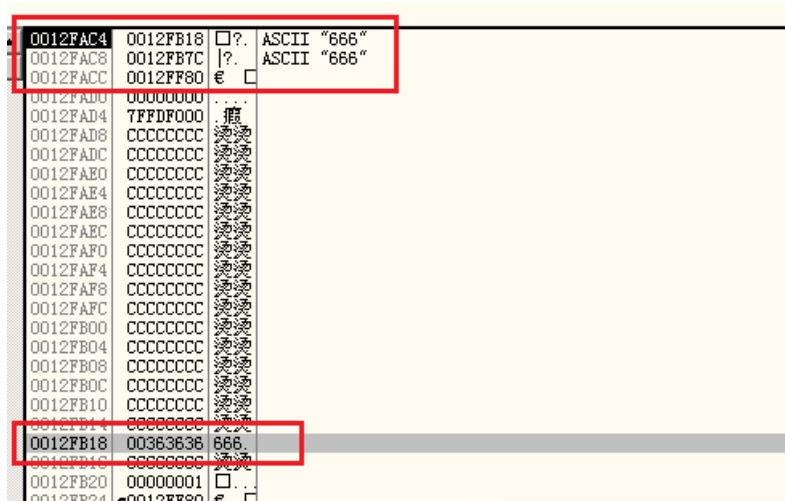
③单步执行

然后点击不进入函数的单步调试按钮(黄色按钮的第四个)。



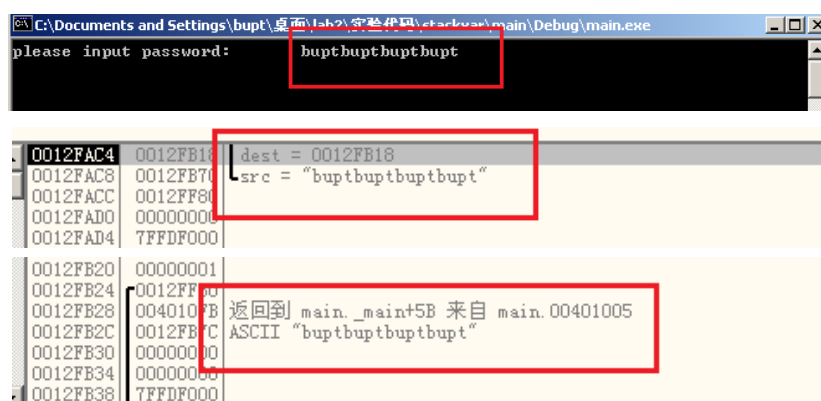
④观察缓冲区中的数据

strcpy 函数执行后，查看缓冲区的内容，输入的是 666，可以看到两个地址中的值被赋值为 666，同时 0012FB18 也为 666。

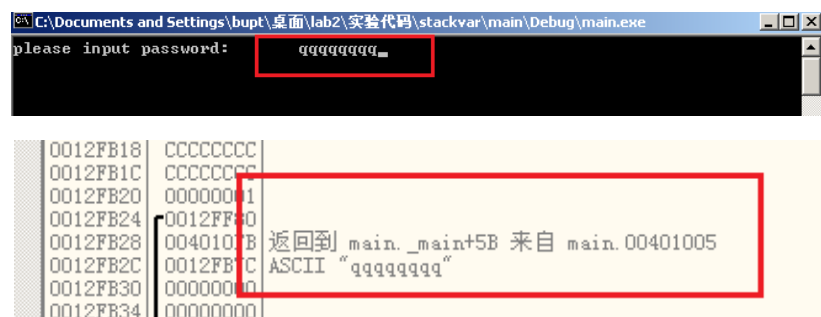


⑤更改输入密码，观察缓冲区中的数据

这次我们输入较长密码“buptbuptbuptbupt”，再进行跟刚才相同的操作，查看缓冲区内容，由于填写的字符串超过位数，已经将邻接的变量覆盖了。



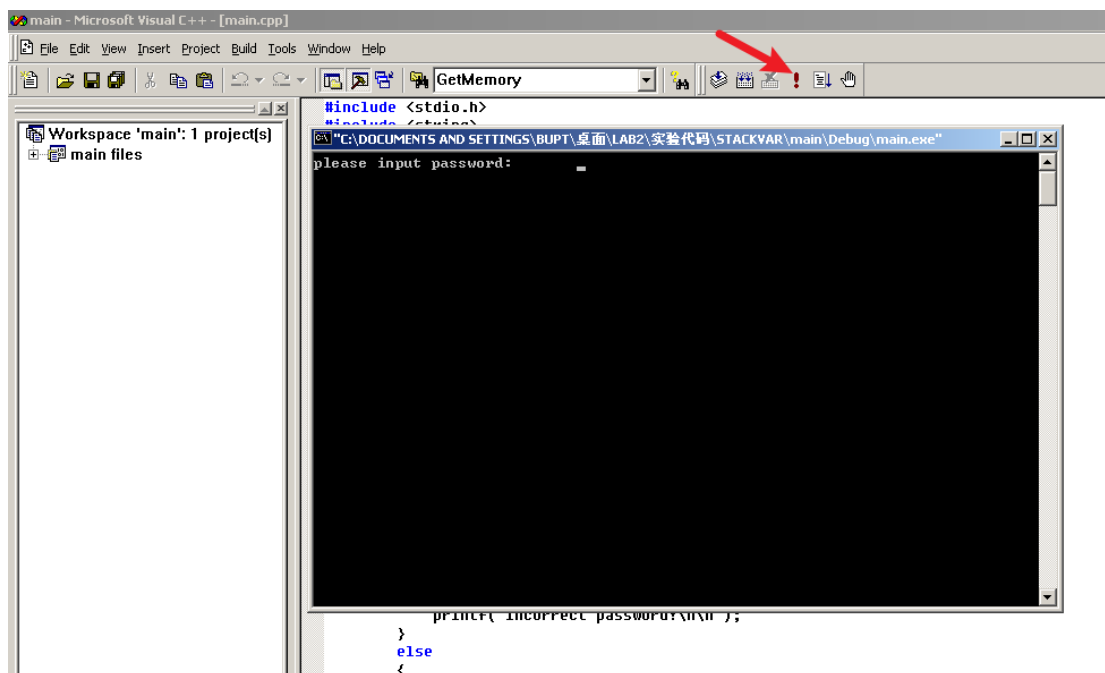
输入“qqqqqqqq”，观察缓冲区中的内容，并且发现程序判断成密码输入正确并自动关闭。



(4) 尝试另一种打开文件调试的方法

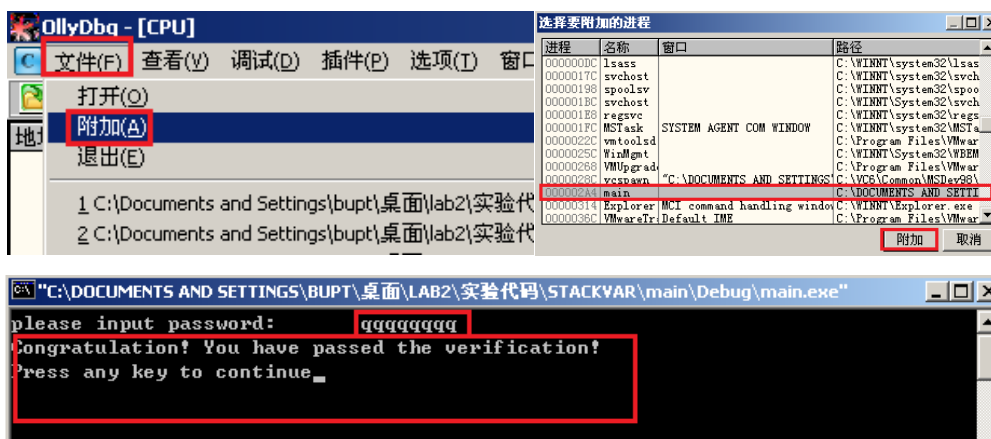
我们如果输入了正确的密码或者 qqqqqqqq，我们发现程序判断正确后则自动关闭，难以看到提示信息。可以尝试以下方法进行调试。

①用 VC 来启动这个工程，然后运行



②调试

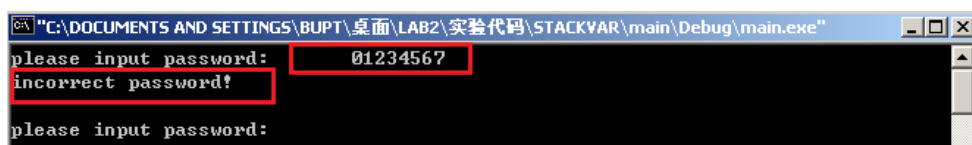
打开 ollydbg, 选择文件->附加->选择进程 main, 这样就可以看到最后的提示信息了。



4、验证补码的问题

当我们输入的密码小于 1234567 时, strcmp 会返回-1 (用补码表示的)。而我们如果输入 01234567 这 8 位字符串, 末尾的 null 只是淹没-1 补码的后两位, 变成 0xFFFFFFFF00, 程序不会朝着我们希望的方向继续下去。

(1) 输入 01234567



(2) 观察 ollydbg 右下角缓冲区中的数据

0012FAC4	0012FB18	dest = 0012FB18
0012FAC8	0012FB7C	src = "01234567"
0012FACC	0012FF80	
0012FB24	0012FF80	
0012FB28	004010FB	返回到 main._main+5B 来自 main.00401005
0012FB2C	0012FB7C	ASCII "01234567"
0012FB30	00000000	

(二) 淹没返回地址

分析如下的变量的排列顺序图，可知只需要先放置 16 个字符串，然后接下来的 4 个字节就能够淹没返回地址，达到控制返回地址的目的。

buffer[0-3]

buffer[4-7]

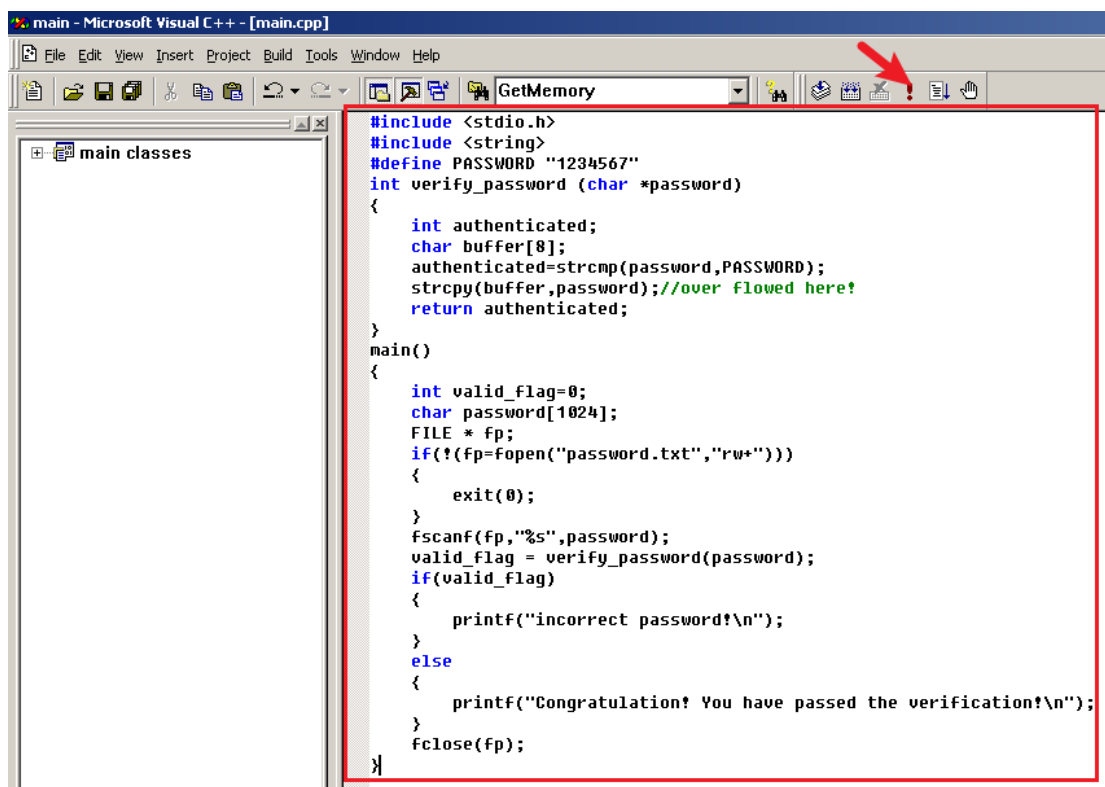
authenticated

EBP

返回地址

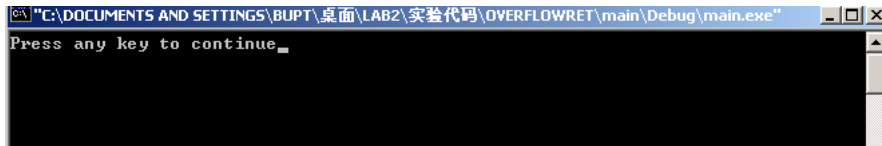
1、新建工程并运行

新建工程和文件，将 main.c 文件的内容复制进新建立的文件中并运行。



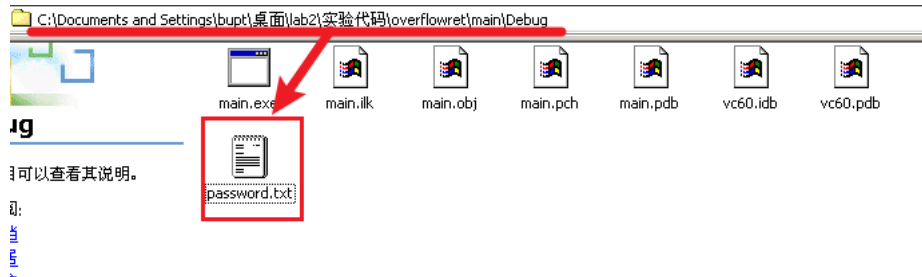
```

#include <stdio.h>
#include <string>
#define PASSWORD "1234567"
int verify_password (char *password)
{
    int authenticated;
    char buffer[8];
    authenticated=strcmp(password,PASSWORD);
    strcpy(buffer,password);//over flowed here!
    return authenticated;
}
main()
{
    int valid_flag=0;
    char password[1024];
    FILE * fp;
    if(!(fp=fopen("password.txt","rw+")))
    {
        exit(0);
    }
    fscanf(fp,"%s",password);
    valid_flag = verify_password(password);
    if(valid_flag)
    {
        printf("incorrect password!\n");
    }
    else
    {
        printf("Congratulation! You have passed the verification!\n");
    }
    fclose(fp);
}
  
```



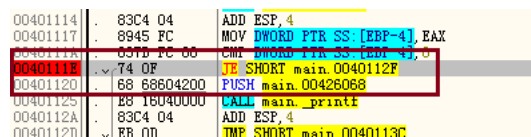
2、新建 password.txt 文件

在上面工程中 DEBUG 文件夹下新建 password.txt 文件。



3、设置断点

用 ollydbg 启动程序，并在跳转条件处 (0040111E) 设置断点。

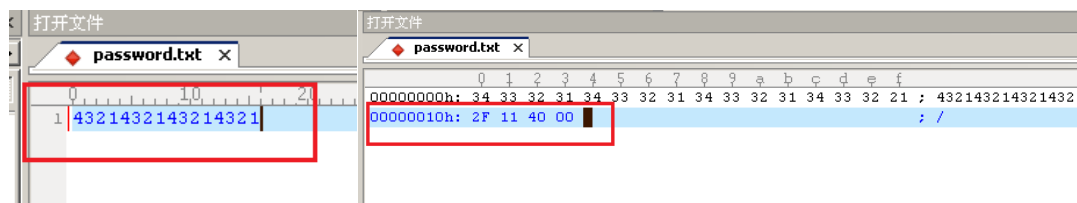


4、调试

分析可得，如果文件中的密码正确的话，程序会跳转到 0040112F。那么我们只需将返回地址修改为 0040112F，就可以使得程序判断密码正确。

(1) 编辑 password.txt 文件

使用 UltraEdit 打开 password.txt 文件，并选择 2 进制编辑模式，前 16 个字节输入 4321432143214321，17-20 字节上填写地址 0040112F 并保存文件。



(2) 运行

运行程序，观察到程序的确提示了密码正确。但是之后由于我们修改了返回地址，导致栈平衡出错，程序崩溃跳出。



三、测试结论

通过本次实验，我对程序堆栈间的调用关系有了更清晰的了解，也熟悉了堆栈溢出的基本原理，并实际动手练习了堆栈溢出利用方法。

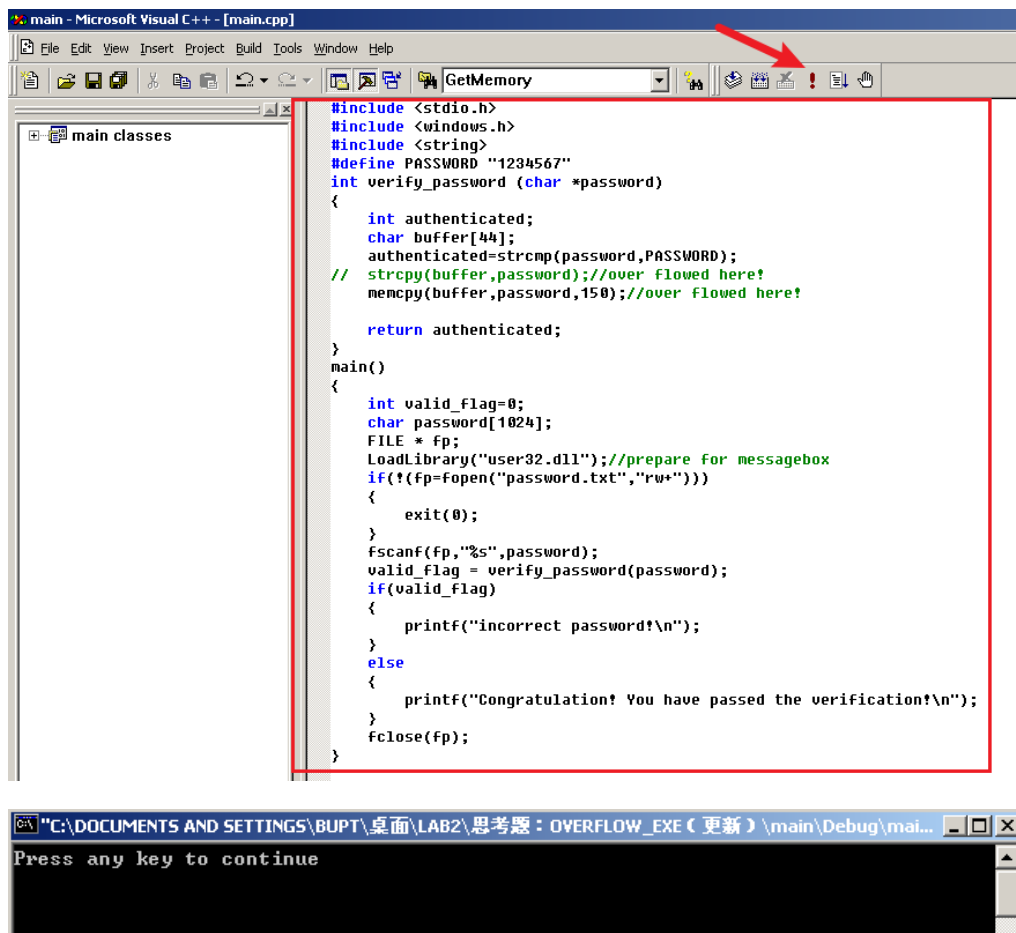
同时我意识到，栈溢出对程序运行的安全性有很大的威胁，如果不对栈进行保护，只要系统中存在栈溢出漏洞，即可令攻击者通过非正当方法获取操作权限。因此，需要对栈中保存数据的可执行权限做严格的管理，并且系统要能判断栈溢出从而采取相应的保护措施。

四、思考题

在不修改源代码的情况下，修改 OverFlow_EXE 程序的流程，通过弧注入的方式，不植入可执行代码，让其调用 MessageBoxA 函数（函数地址：0x77E23D68）弹出对话框（对话框显示 bupt），之后调用 ExitProcess 函数（函数地址：0x77E7B0BB）退出进程。

1、新建工程

将“思考题：overflow_exe（更新）”文件夹下的 main.cpp 中的内容复制并粘贴到新建立的工程文件中并运行。



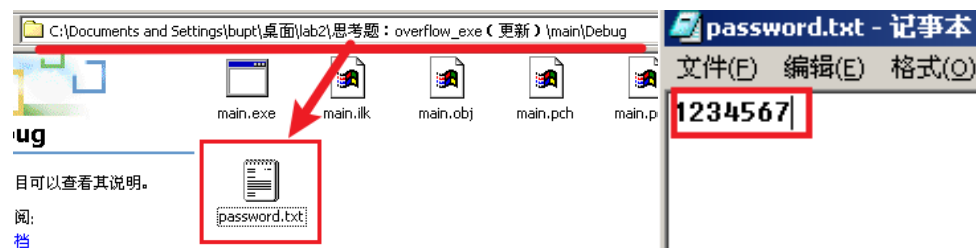
The screenshot displays the Microsoft Visual C++ IDE. The top window shows the source code of main.cpp, which includes headers for stdio.h, windows.h, and string.h, and defines a PASSWORD constant. The code implements a password verification function and a main function that reads a password from a file and calls verify_password. A red box highlights the memory copy operation in verify_password, and a red arrow points to the 'Run' button in the toolbar. The bottom window shows the program's execution output, which is a black console window with the text 'Press any key to continue'.

```
#include <stdio.h>
#include <windows.h>
#include <string.h>
#define PASSWORD "1234567"
int verify_password (char *password)
{
    int authenticated;
    char buffer[44];
    authenticated=strcmp(password,PASSWORD);
    // strcpy(buffer,password);//over flowed here!
    memcpy(buffer,password,150);//over flowed here!
    return authenticated;
}
main()
{
    int valid_flag=0;
    char password[1024];
    FILE * fp;
    LoadLibrary("user32.dll");//prepare for messagebox
    if(! (fp=fopen("password.txt","rw+")))
    {
        exit(0);
    }
    fscanf(fp,"%s",password);
    valid_flag = verify_password(password);
    if(valid_flag)
    {
        printf("incorrect password!\n");
    }
    else
    {
        printf("Congratulation! You have passed the verification!\n");
    }
    fclose(fp);
}
```

Press any key to continue

2、新建 password.txt 文件

在上述工程 Debug 文件夹中新建 password.txt，并暂时写入 1234567 作为密码。



3、调试

(1) 设置断点

在 ollydbg 中打开程序，并在 memcpy 函数处 (00401069 处) 设置断点。

00401061	8B4D 08	MOV ECX, DWORD PTR SS:[EBP+8]	
00401064	51	PUSH ECX	src
00401065	8D55 D0	LEA EDX, DWORD PTR SS:[EBP-30]	
00401068	52	PUSH EDX	dest
00401069	E8 52020000	CALL main._memcpy	memcpy
0040106E	83C4 0C	ADD ESP, 0C	
00401071	8B45 FC	MOV EAX, DWORD PTR SS:[EBP-4]	
00401074	5F	POP EDI	
00401075	5E	POP ESI	

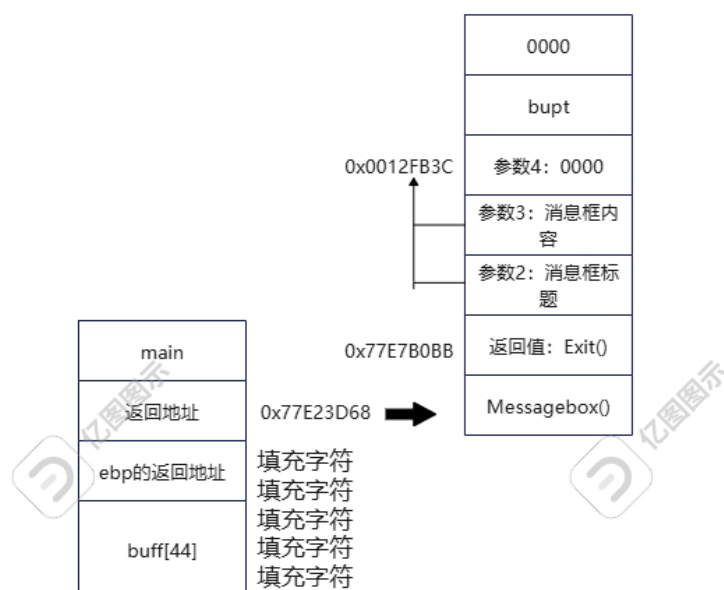
(2) 观察缓冲区中的数据

运行程序至断点处，再步过函数运行 memcpy 函数，此时观察 ollydbg 右下角缓冲区中的数据。可以看到保存返回地址的地址为 0012FB24，向下步过可以看到字符存储地址为 0012FAF0。

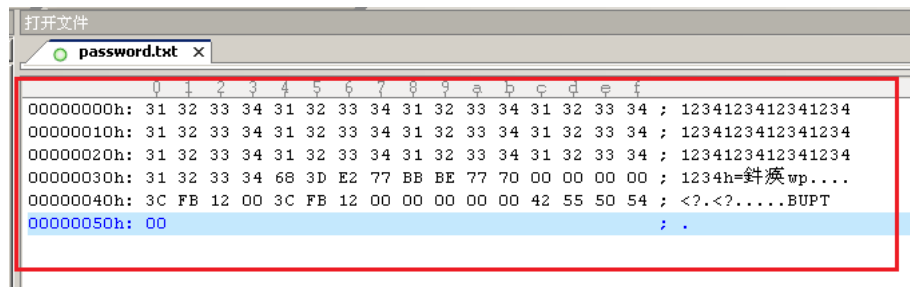
0012FB20	0012FB80	返回到 OverFlow._main+88 来自 OverFlow.00401005	0012FAEC	CCCCCCCC
0012FB24	00401128		0012FAF0	34333231
0012FB28	0012FB7C	ASCII "1234567"	0012FAF4	00373635
0012FB2C	00000000			
0012FB30	00000000			
0012FB34	7FFDF000			

(3) 弧注入

弧注入的栈结构如下：



两个地址之间的距离为 13×4 ，其为需要填充的长度，MessageBoxA 地址为 0x77E23D68，ExitProcess 地址为 0x77E7B0BB，输出参数为 bupt，由此可以通过 UltraEdit 编写 password.txt 文件，在其中输入 payload。



(4) 运行

运行程序即可看到窗口弹出。

